



Domain 3: Continuous Improvement for Existing Solutions



Overview

The third domain of the AWS Certified Solutions Architect Professional SAP-C02 exam focuses on continuously improving your current cloud solutions by leveraging on new AWS services and features. Each month, AWS releases a set of updates on their services which are most of the time new features but can sometimes be freshly minted services themselves. By keeping yourself up-to-date with the latest technologies and best practices available, you can help you and your customers improve the existing cloud architectures that you manage.

You should know how to properly analyze and refactor certain components of your current cloud solutions to further improve your services. This entails the use of automation, deployment tools, data replication methods, scaling methodologies, logging & monitoring strategies, and other components in improving your existing stack.

This domain covers a quarter (25%) of the questions in the actual SAP-C02 exam and is the third largest domain. The topics for this domain are focused on these task statements.

- Determining a strategy to improve overall operational excellence
- Determining a strategy to improve security.
- Determining a strategy to improve performance
- Determining a strategy to improve reliability.
- Identifying opportunities for cost optimizations.

Using Amazon Cognito for Web App Authentication

Instead of building your own user management system for your websites and web applications, AWS offers a much simpler alternative with Amazon Cognito. Amazon Cognito is a service that allows you to add user sign-up, sign-in, and access control to your web and mobile apps. You can construct your user pool or use social identity providers to provide users a convenient method of signing up and logging in. Amazon Cognito also supports enterprise identity providers such as Microsoft Active Directory using SAML.

Amazon Cognito offers two types of pools for your business applications – **user pools** and **identity pools**. The main difference between the two is that user pools are used for authentication (identify verification) while identity pools are for authorization (access control). For authentication, Amazon Cognito uses multiple identity management standards including OpenID Connect, OAuth 2.0, and SAML 2.0.

Users Pools

With a user pool, your users can sign in through the user pool or federate through a third-party identity provider. It essentially acts as a directory. Use cases include:

- Be able to add sign-up and sign-in features for your app.



- Be able to access and manage user data.
- Be able to track user device, location, and IP address, and adapt to sign-in requests of different risk levels.
- Be able to use a custom authentication flow for your app.
- Be able to access resources with Amazon API Gateway and AWS Lambda.

Identity Pools

Identity pools provide tokens that can be exchanged for temporary AWS credentials in AWS STS after a successful authorization. The permissions for each user's credentials are controlled through IAM roles that you create. You can use identity pools to create unique identities for users and give them access to your AWS services. Use cases include:

- Giving your users access to AWS resources, such as an Amazon S3 bucket or an Amazon DynamoDB table.
- Generating temporary AWS credentials for unauthenticated users.

Users Pools + Identity Pools

There is no rule stating that you cannot use these two services together. An example of a use case is when you want to manage your users in Amazon Cognito and you would like to provide them temporary access to your AWS services. After a successful user pool authentication, the user's app will receive user pool tokens from Amazon Cognito. The user can then exchange them for temporary access to AWS services with an identity pool.

AWS AppSync (Newer service than Cognito Sync)

AWS AppSync is a service that lets you manage and synchronize mobile app data in real time across different devices and users, but still allows the data to be accessed and altered when the mobile device is offline. To tighten security around using AWS AppSync, you can grant your users access to AppSync resources with tokens from a successful Amazon Cognito authentication.

Scenario: Accessing Resources with Amazon API Gateway and AWS Lambda After Sign-in

You should make sure users accessing your API through Amazon API Gateway are authorized to do so. You can configure API Gateway to validate the tokens from a successful user pool authentication in Amazon Cognito, and use them to grant your users access to resources including Lambda functions, or your own API. Token verification is usually performed by an Amazon Cognito authorizer Lambda function.

References:

<https://aws.amazon.com/cognito/>

<https://aws.amazon.com/premiumsupport/knowledge-center/cognito-user-pools-identity-pools/>



<https://aws.amazon.com/appsync/>

Using AWS Systems Manager for Patch Management

We patch servers regularly during each of our maintenance periods to make sure that our operating systems are always kept up-to-date with the latest bug fixes and security fixes. This is especially crucial for production workloads since there are always new security vulnerabilities being discovered each day, and most, if not all of them, are too risky to simply leave unresolved. But of course, patching activity also presents its own set of challenges, particularly when there are multiple servers involved and each have a different patch baseline. Manually connecting to your instances and running Windows Update or *sudo yum update* is not feasible, so you will need to automate your patching activities. The second challenge here is coordinating your patching window with your server availability. There are many approaches to solving these challenges, but for this section we will be focusing on using AWS Systems Manager.

Note: Since you are letting AWS Systems Manager handle your instances, you will need to assign the appropriate IAM role to your instances that would grant permissions for AWS Systems Manager to perform patching and other related tasks. You also need to make sure that your instances have SSM Agent installed and the agents are able to communicate back with AWS Systems Manager. One way to verify this is to go to AWS Systems Manager Managed Instances and check if your desired instances are in the list.

There are two key services under AWS Systems Manager that we will use to build our fully-automated patching solution, namely:

- 1) Patch Manager
- 2) Maintenance Windows

Patch Manager automates the process of patching managed instances with both security related and other types of updates. You can use Patch Manager to apply patches for both operating systems and applications. You can patch fleets of EC2 instances or your on-premises servers and virtual machines (VMs) by the type of operating system. You can also scan instances to see only a report of missing patches, or you can scan and automatically install all missing patches.

Patch Manager uses **patch baselines** to let you specify which patches to install and apply a delay after patches are released before auto-approving them. AWS already provides you a set of preconfigured patch baselines for different OS types, but you can also configure your own if you wish.



AWS Systems Manager > Patch Manager > Baseline ID: pb-09ca3fb51f0412ec3

Baseline ID: pb-09ca3fb51f0412ec3

Edit Delete Actions ▼

Description

Baseline ID	Baseline name
arn:aws:ssm:us-east-1:075727635805:patchbaseline/pb-09ca3fb51f0412ec3	AWS-DefaultPatchBaseline
Description	Operating system
Default Patch Baseline Provided by AWS.	Windows Server
Default baseline	Patch groups
Yes	-
Created date (UTC)	Modified date (UTC)
Tue, 01 May 2018 17:13:20 GMT	Tue, 01 May 2018 17:13:20 GMT

Approval rules

Product	Classification	Severity	Auto approval delay	Approve Until Date	Compliance reporting
-	CriticalUpdates,SecurityUpdates	Critical,Important	Wait 7 days before approving	-	Unspecified

Approval rules for Microsoft applications

Product family	Product	Classification	Severity	Auto approval delay	Approve Until Date	Compliance reporting
No rules.						

▼ Patch exceptions

Approved patches	Rejected patches
-	-
Approved patches compliance level	Rejected patches action
Unspecified	Allow as dependency

Maintenance Windows, on the other hand, lets you define a **schedule** for when to perform potentially disruptive actions on your instances such as patching an operating system, updating drivers, or installing software or patches. Each maintenance window has a schedule, a maximum duration, a set of registered targets (the instances or other AWS resources that are acted upon), and a set of registered tasks.

In summary, a maintenance window for patching works like this:

1. You create a maintenance window with a schedule for your patching operations.
2. Then choose the targets for the maintenance window by specifying either a **Patch Group** tag or your own tag key, or by choosing the instances manually.
3. Finally, create a new maintenance window task, and specify the **AWS-RunPatchBaseline** document or any other document you would like to use for your patching operation.



These steps can also be done conveniently in the Patch Manager Window.

AWS Systems Manager > Patch Manager > Configure patching

Configure patching

Instances to patch

How do you want to select instances?

- ☒ Enter instance tags
- ☐ Select a patch group
- ☐ Select instances manually

Instance tags
Specify one or more instance tag key/value pairs to identify the instances you want to patch.

Enter a tag key and optional value applied to the instances you want to target, and then choose **Add**

Patching schedule

How do you want to specify a patching schedule?

- ☒ Select an existing Maintenance Window
- ☐ Schedule in a new Maintenance Window
- ☐ Skip scheduling and patch instances now

Maintenance Window
Select a [Maintenance Window](#)

Patching operation

- ☒ **Scan and install**
Scans each target instance and compares its installed patches with the list of approved patches in the patch baseline. Downloads and installs all approved patches that are missing from the instance.
- ☐ **Scan only**
Scans each target instance and generates a list of missing patches for you to review.

► **Additional settings**



When creating a schedule for your maintenance window, you have two ways to build it.

- a) Scheduling via cron expression - With cron, you can schedule your window at a specific rate (for example, every 30 minutes or every hour) or at a specific day and time (for example, run everyday at 12 am or run every Sunday at 1 am). You can also specify at what date should your tasks start executing and until when. Lastly, you can change your preferred time zone in case you follow a different one.
- b) Scheduling via rate expression - With rate, your available options are similar to cron except that you cannot specify at which day and time you want your tasks to execute. As the name implies, you can only specify rates (for example, every 30 minutes, every 1 hour, or everyday)

i Solutions Architect Professional Exam Notes:

So what should I look out for before choosing these two services for an item in the exam?

The answer is convenience. Remember that as a Solutions Architect, you are supposed to build solutions that would make your business operations more convenient and simple. Patch Manager and Maintenance Windows help you out in this area since they are less disruptive than other options, while making sure the job gets done. You are offered a lot of flexibility in the configurations for the scheduling and the patch baselines. And since this is automation, your solution can be reused multiple times and the results will always be predictable.

Some other options that might throw you off include Amazon EventBridge for the scheduling or Systems Manager State Manager for patch compliance. If you have a good understanding of these two services then you should also know why they aren't the best choices. Scripting your own cron scheduler and patching automation are also ruled out since they are not the most convenient to do and maintain.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/sysman-patch-mw-console.html>
[Patching Windows Servers using AWS Systems Manager](#)



Implementing CI/CD using AWS CodeDeploy, AWS CodeBuild, and AWS CodePipeline

It is highly likely that in your exam, you will also encounter scenarios involving CI/CD. What you need to learn for these kinds of scenarios are the CI/CD tools in AWS and how to build your own pipelines using them. To start with, we will first briefly define each AWS CI/CD tool and in what situations will you use them for.

1) AWS CodeDeploy

AWS CodeDeploy is a fully managed deployment service that automates software deployments to Amazon EC2, AWS Fargate, AWS Lambda, and your on-premises servers. The steps for initiating a deployment involves:

- Choosing your compute platform for your deployment
- Creating a deployment group
- Providing CodeDeploy with the necessary permissions via service role
- Specifying the targets for that compute platform
- Configuring deployment settings such as choosing your deployment strategy, setting up alarms and notifications, creating deployment triggers, defining rollback settings and adding tags.

AWS CodeDeploy offers multiple deployment strategies for each compute platform. For Amazon EC2 instances and on-premises servers, you can select if you would like to use **in-place** or **blue-green** deployment. An in-place deployment updates your instances/servers right as they are. The application on each instance in the deployment group is stopped, the latest application revision is installed, and the new version of the application is started and validated. These updates can be deployed to your instances/servers one at a time, in batches, or all at once. In-place deployments can be used if your applications have redundant copies to which traffic can failover to, or if the applications being updated are not critical to the overall availability of your system.

Blue-green deployment is a better strategy for workloads that cannot tolerate interruptions, such as an active website. Furthermore, transitioning to the new application version is more gradual and controlled, and rolling back to a previous working version is easier and quicker. In a blue-green deployment, the instances in a deployment group are replaced by a different set of instances containing your updates. New instances will be registered to your load balancer so that it can start accepting traffic, while old instances are deregistered. Deregistered instances can be kept alive for rollback scenarios, or can be terminated immediately. Similarly, these updates can be deployed to your instances/servers one at a time, in batches, or all at once.

Deployment type

Choose how to deploy your application

☒ **In-place**
 Updates the instances in the deployment group with the latest application revisions. During a deployment, each instance will be briefly taken offline for its update

☐ **Blue/green**
 Replaces the instances in the deployment group with new instances and deploys the latest application revision to them. After instances in the replacement environment are registered with a load balancer, instances from the original environment are deregistered and can be terminated.

Environment configuration

Select any combination of Amazon EC2 Auto Scaling groups, Amazon EC2 instances, and on-premises instances to add to this deployment

☐ Amazon EC2 Auto Scaling groups

☐ Amazon EC2 instances

☐ On-premises instances

Deployment settings

Deployment configuration

Choose from a list of default and custom deployment configurations. A deployment configuration is a set of rules that determines how fast an application is deployed and the success or failure conditions for a deployment.

CodeDeployDefault.OneAtATime ▼

or

Create deployment configuration

Figure: In Place Deployment in AWS CodeDeploy

Deployment type

Choose how to deploy your application

☐ **In-place**
 Updates the instances in the deployment group with the latest application revisions. During a deployment, each instance will be briefly taken offline for its update

☒ **Blue/green**
 Replaces the instances in the deployment group with new instances and deploys the latest application revision to them. After instances in the replacement environment are registered with a load balancer, instances from the original environment are deregistered and can be terminated.

Environment configuration

Specify the Amazon EC2 Auto Scaling groups or Amazon EC2 instances where the current application revision is deployed.

☒ **Automatically copy Amazon EC2 Auto Scaling group**
 Provision an Amazon EC2 Auto Scaling group and deploy the new application revision to it. AWS CodeDeploy will create the Auto Scaling group by copying the one you specify here.

☐ **Manually provision instances**
 I will specify here the instances where the current application revision is running. I will specify the instances for the replacement environment when I create a deployment.

Choose the Amazon EC2 Auto Scaling group where the current application revision is deployed.

Deployment settings

Traffic rerouting

☒ Reroute traffic immediately
 ☐ I will choose whether to reroute traffic

Choose whether instances in the original environment are terminated after the deployment is succeeds, and how long to wait before termination.

☒ Terminate the original instances in the deployment group
 ☐ Keep the original instances in the deployment group running

Days
 Hours
 Minutes

Deployment configuration
 Choose from a list of default and custom deployment configurations. A deployment configuration is a set of rules that determines how fast an application is deployed and the success or failure conditions for a deployment.

or

Load balancer

Select a load balancer to manage incoming traffic during the deployment process. The load balancer blocks traffic from each instance while it's being deployed to and allows traffic to it again after the deployment succeeds.

☒ Enable load balancing

☒ **Application Load Balancer or Network Load Balancer**
☐ Classic Load Balancer

Choose a target group

Figure: Blue Green Deployment in AWS CodeDeploy

For Lambda and ECS/Fargate platforms, you only have blue-green deployment as your available deployment strategy. You can have your updates deployed all at once, gradually in a linear fashion (meaning a percentage of the traffic is shifted from the old to the new at a rate until all traffic is shifted), or in a two-step order using canary (meaning a small percentage of traffic is given to the new, and the rest of the traffic is shifted to the new environment after a specified wait time)

Create deployment group

Application

Application
testlambda
Compute type
AWS Lambda

Deployment group name

Enter a deployment group name

100 character limit

Service role

- CodeDeployDefault.LambdaAllAtOnce
- CodeDeployDefault.LambdaLinear10PercentEvery1Minute
- CodeDeployDefault.LambdaLinear10PercentEvery2Minutes
- CodeDeployDefault.LambdaLinear10PercentEvery3Minutes
- CodeDeployDefault.LambdaLinear10PercentEvery10Minutes
- CodeDeployDefault.LambdaCanary10Percent5Minutes
- CodeDeployDefault.LambdaCanary10Percent10Minutes
- CodeDeployDefault.LambdaCanary10Percent15Minutes
- CodeDeployDefault.LambdaCanary10Percent30Minutes
- CodeDeployDefault.LambdaAllAtOnce

s to your target instances.

configuration is a set of rules that determines how fast

or [Create deployment configuration](#)

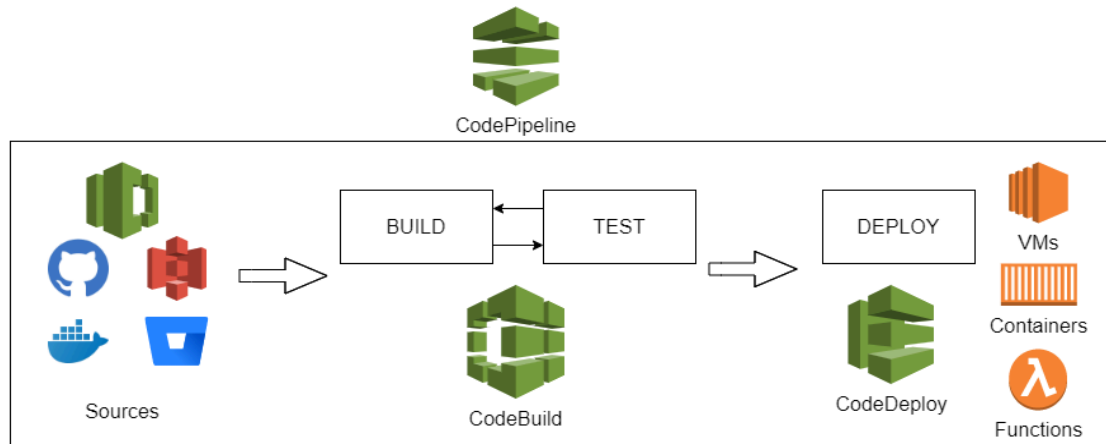
Figure: Lambda Deployment in AWS CodeDeploy

Reference:

<https://aws.amazon.com/codedeploy/>

2) AWS CodeBuild

AWS CodeBuild is a continuous integration service that **compiles source code, runs tests, and produces software packages** that are ready to deploy. AWS CodeBuild does not retrieve source code from your repositories, nor does it deploy packages to your machines.



AWS CodeBuild is often used together with a pipeline, either with AWS CodePipeline or third party software such as Jenkins.

To use CodeBuild, you first create a project. In it, you specify the source of your builds, the environment and operating system you want your code to compile and get tested in, an IAM service role to allow CodeBuild to run, a *buildspec* YAML file to define how to compile and test your code, and lastly, an S3 bucket to store your *artifact* or “final product”. You can also store CodeBuild logs in Cloudwatch logs or in S3. Specifying an S3 bucket for artifacts is not required if you are only conducting testing or if you are compiling a Docker image, which of course should be uploaded to a Docker repository.

References:

<https://aws.amazon.com/codebuild>
[Setting up CI/CD for containers](#)

3) AWS CodePipeline

AWS CodePipeline is a continuous delivery service that helps you automate the build, test, and deploy phases of your release process every time there is a code change. To understand AWS CodePipeline better, we will need to define a few terminologies:



- A *pipeline* is a workflow construct that describes how software changes go through a release process. You define the workflow with a sequence of stages and actions.
- A *stage* is a group of one or more actions. A pipeline can have two or more stages.
- An *action* is a task performed on a revision. Pipeline actions occur in a specified order, in serial or in parallel, as determined in the configuration of the stage.
- A *revision* is a change made to the source location defined for your pipeline. It can include source code, build output, configuration, or data.
- The stages in a pipeline are connected by *transitions*. Revisions that successfully complete the actions in a stage will be automatically sent on to the next stage as indicated by the transition.
- When an action runs, it acts upon a file or set of files called *artifacts*. These artifacts can be worked upon by later actions in the pipeline.

Reference:

<https://aws.amazon.com/codepipeline/>

<https://aws.amazon.com/blogs/devops/build-a-continuous-delivery-pipeline-for-your-container-images-with-amazon-ecr-as-source/>

4) Amazon ECS / AWS Fargate

Amazon ECS and AWS Fargate are both compute services for containers. It is common for containers to be used in a CI/CD deployment. In the exam, what you have to know is that an Amazon ECS and AWS Fargate deployment requires four (4) components:

- An ECR Repository where you will store versioned container images (source of your pipeline).
- An ECS Cluster which will be your cluster of container instances. This will include a load balancer and auto scaling configurations.
- ECS Task Definition which specifies your container image and environment configurations.
- ECS Service which specifies how your task definition will be deployed onto underlying compute resources.

There are also two IAM roles that you need to distinguish: The ECS **task execution role** grants the Amazon ECS container and Fargate agents permission to make AWS API calls on your behalf. The ECS **task role** grants applications running in your containers permission to make AWS API calls.

References:

<https://aws.amazon.com/ecs/>

<https://aws.amazon.com/fargate/>

5) AWS X-Ray

AWS X-Ray provides an end-to-end view of requests as they travel through your application, and shows



a map of your application's underlying components. With X-Ray, you can understand how your application and its underlying services are performing to identify and troubleshoot the root cause of performance issues and errors. In the exam, if the scenario presents multiple components in an application or a complex microservice architecture (e.g. APIs, functions, containers, etc), use AWS X-Ray to pinpoint and debug HTTP errors.

You can get started with X-Ray by including the X-Ray language SDK in your application and installing the X-Ray agent. X-Ray can be used with distributed applications of any size to trace and debug both synchronous requests and asynchronous events. You can use X-Ray with applications running on EC2, ECS, Lambda, Amazon SQS, Amazon SNS, and Elastic Beanstalk.

Reference:

<https://aws.amazon.com/xray/>

Using Federation to Manage Access

When you are in an organization with multiple users and multiple accounts, one way to provide your users access to AWS in a secure and centrally manageable manner is through federation. You can use two AWS services to federate into AWS: AWS IAM Identity Center (formerly AWS SSO) and AWS IAM. Use the AWS IAM Identity Center to help you define federated access permissions for your users based on their group memberships in a single centralized directory. If you use multiple directories, or want to manage the permissions based on user attributes, use AWS IAM instead.

AWS IAM Identity Center (formerly AWS SSO)

AWS IAM Identity Center works with identity provider (IdP) services such as Okta Universal Directory or Azure Active Directory via the SAML 2.0. You can add any AWS account managed using AWS Organizations to AWS IAM Identity Center, but you need to enable all features in your organizations first. The AWS IAM Identity Center service leverages IAM permissions and policies for federated users and roles to help you manage federated access centrally across all AWS accounts in your AWS Organization. With AWS IAM Identity Center, you can assign permissions based on the group membership in your IdP's directory, and then control the access for your users by simply modifying users and groups in the IdP.

You can also control who can have access to your cloud applications as it can securely communicate with your applications through a trusted relationship between the AWS IAM Identity Center and the application's service provider. This trust is created when you add the application from the AWS IAM Identity Center console and configure it with the appropriate metadata for both the IAM Identity Center and the service provider. Take note that this service supports only SAML 2.0-based applications, so using OIDC-based applications will not work.

AWS IAM

AWS IAM allows you to enable a separate SAML 2.0 or an Open ID Connect (OIDC) IdP for each AWS account you manage and use federated user attributes for access control. Instead of creating IAM users, you can use IAM identity providers to manage your user identities outside of AWS and give these external user identities permissions to use AWS resources in your account. This is useful if your organization already has its own identity system, such as a corporate user directory. It is also useful if you are creating a mobile app or web application that requires access to AWS resources.

To use an IdP, you create an IAM identity provider entity to establish a trust relationship between your AWS account and the IdP.

- With **web identity federation**, you don't need to create custom sign-in code or manage your own user identities. Instead, users of your app can sign in using an OpenID Connect (OIDC)-compatible IdP. They will receive an authentication token, and then your application calls the `AssumeRoleWithWebIdentity` API to exchange that token for temporary security credentials in AWS. The credential is mapped to an IAM role with permissions to use the resources in your AWS account. For convenience, use Amazon Cognito as your identity broker for almost all web identity federation scenarios.
- With **SAML 2.0-based federation**, a user in your organization requests authentication from your organization's IdP through an app. The IdP authenticates the user against your organization's identity store. Then the IdP constructs a SAML assertion with information about the user and sends the assertion to the app. The app calls the AWS STS `AssumeRoleWithSAML` API, passing the ARN of the SAML provider, the ARN of the role to assume, and the SAML assertion from IdP. The API response to the app includes temporary security credentials, which the user can use to access your AWS resources.

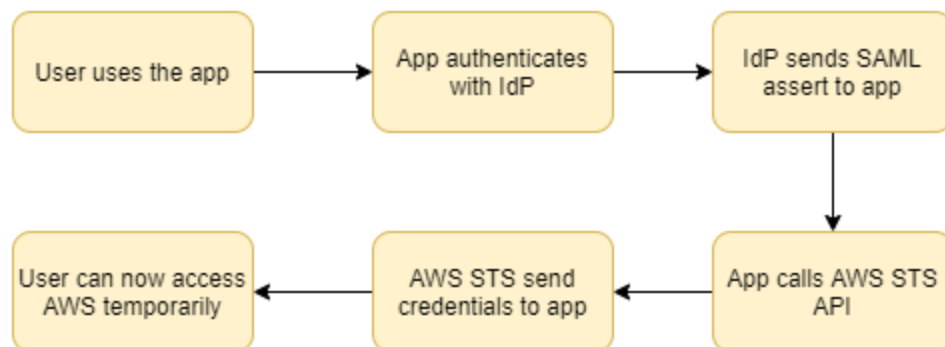


Figure: AWS IAM SAML federation

References:

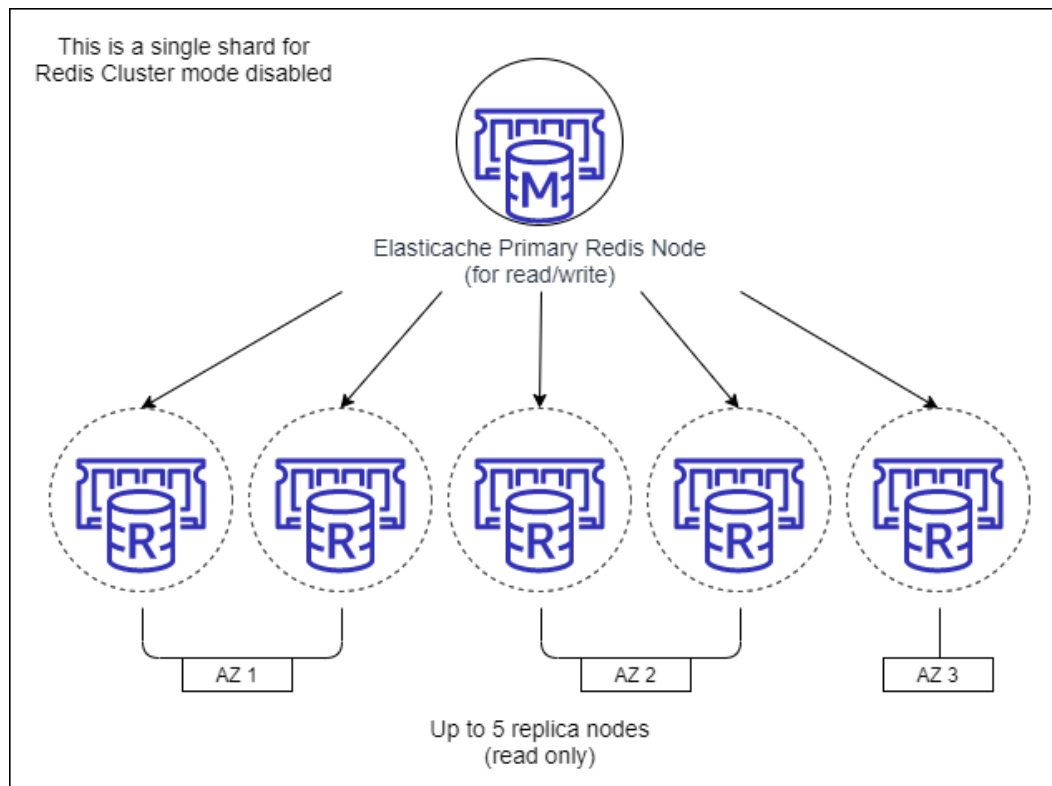
<https://aws.amazon.com/identity/federation/>
<https://docs.aws.amazon.com/singlesignon/latest/userguide/what-is.html>
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_providers.html

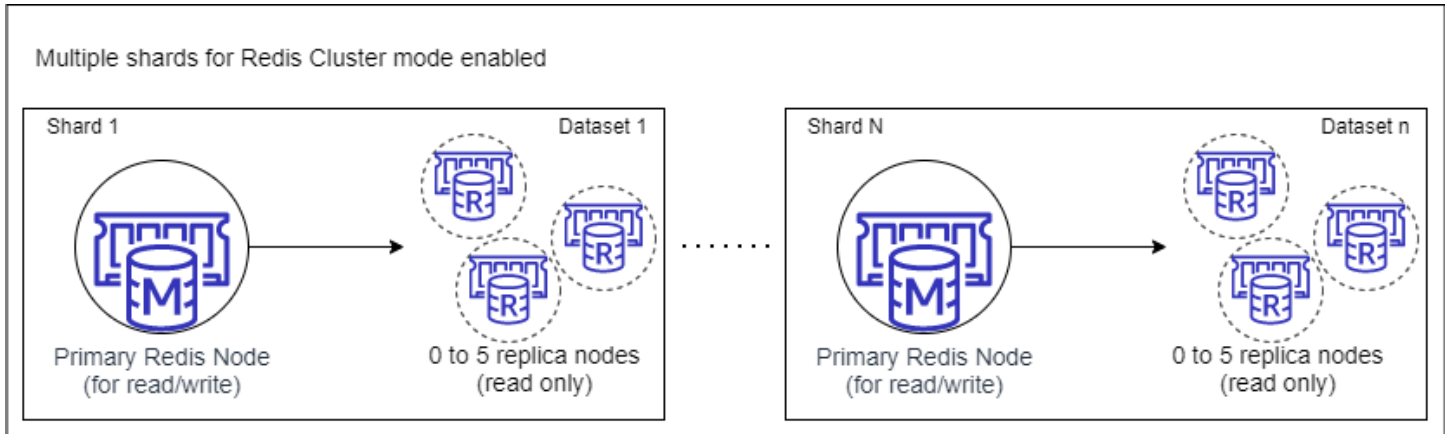
Setting Up a Fault Tolerant Cache Layer with Amazon ElastiCache

When adding a caching layer to your infrastructure, similar to databases, you should also make sure that your cache is highly available to avoid any issues. If you only deploy single nodes for example, a sudden outage or reboot on this node can incur large amounts of data loss. Your applications will also take a hit since performance will be greatly impacted, and unless you have configured a fault-tolerant system, might even cause downtime. To protect your cache from a total outage, you can configure **Replication Groups** or set up **Append Only Files**.

Redis Replication Groups

In Amazon ElastiCache for Redis, you can have 2 to 6 nodes in a cluster where 1 - 5 nodes are read-only replicas. In this scenario, if one node were to fail, you do not lose all your data because of the replication. But since the replication mode is asynchronous, some data may be lost if it is the primary read/write node that fails. Recall that for Redis cluster mode disabled, clusters always have one shard. On the other hand, Redis cluster mode enabled clusters can have up to 90 shards. Redis cluster mode enabled provides you the flexibility to create a cluster with your desired number of shards and number of replicas with up to 5 replicas (as long as the total is 90 nodes). If the cluster with replicas has Multi-AZ enabled and the primary node fails, the primary fails over to a read replica. **Multi-AZ is required for all Redis (cluster mode enabled) clusters!**





When a failover does occur, ElastiCache also propagates the DNS name of the promoted replica. No endpoint change is required in your application if it is using the primary endpoint. Applications using individual endpoints need to change the read endpoint of the replica promoted to primary to the new replica's endpoint.

Append Only Files

If you cannot use replication groups due to some constraint, but still need data durability, you may use Redis append-only file feature (AOF). When this feature is enabled, your ElastiCache Redis node writes all of the commands that change cache data to an append-only file. If the node is rebooted, the AOF is "replayed", much like a database recovery process. This ensures that your cache data remains intact.

To enable AOF for a cluster running Redis, you must create a parameter group with the *appendonly* parameter set to yes. You then assign that parameter group to your cluster. You can also modify the *appendfsync* parameter to control how often Redis writes to the AOF file.

Between the two fault tolerance solutions, the better option to implement would be the replication group with multi-AZ.

References:

<https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/Replication.html>
<https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/RedisAOF.html>

Improving the Cache Hit Ratio of your CloudFront Distribution

One of the main purposes of using CloudFront is to reduce the number of requests that your origin server must respond to directly. CloudFront caching allows you to serve objects from CloudFront edge locations, which are closer to your users. This effectively reduces the load on your origin server and reduces latency. If you notice that your CloudFront distribution is not doing a good job caching your objects, and that your origin server is responding too frequently, you can optimize your cache settings to encompass a larger subset of cacheable objects.

The proportion of requests that are served from caches to all requests is called the *cache hit ratio*. There are a number of changes you can do to improve your cache hit ratio.

- 1) Increase the duration that your objects stay cached in CloudFront edge locations. You can configure your origin to add a Cache-Control max-age header to your objects, and specify the longest practical value for max-age. The shorter the cache duration, the more frequently CloudFront checks if the object has changed to get the latest version.
- 2) Configure CloudFront to forward only the query string parameters for which your origin will return unique objects.
- 3) Configure CloudFront to forward only specific cookies instead of all cookies to your origin. Create separate cache behaviors for static and dynamic content, and forward cookies to your origin only for dynamic content.
- 4) Configure CloudFront to forward and cache objects based on specific headers only instead of forwarding and caching objects based on all headers.
- 5) Remove Accept-Encoding Header when compression is not needed. When you use this configuration, CloudFront removes the header from the cache key and doesn't include the header in origin requests.
 - Header name: Accept-Encoding
 - Header value: (Keep blank)

To check if any of these changes has helped you improve your cache hit ratio, you may visit the CloudFront Cache Statistics Reports page and review the metrics.

References:

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/ConfiguringCaching.html>
<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/cache-statistics.html>



Other Ways of Combining Route 53 Records for High Availability and Fault Tolerance

To build a fully highly-available and fault tolerant infrastructure, it's not only your EC2 instances and RDS databases that you should worry about. You also need to make sure that your users are properly routed to a working origin in case of a failover with least amount (and if possible, zero) downtime. Protection from a single point of failure typically includes having health checks continuously monitoring your endpoints, and distributing your endpoints in different locations. In AWS, you get less headaches if you don't place all your eggs in one basket.

Though Route 53 has made failover routing possible with active-active and active-passive failover solutions, it is not always the case that failover records are the best to use for your environment. For example, you might want to route your users to the servers closest to them, or serve specific content based on where your customers are. In these scenarios, you might consider other, more beneficial routing policies.

Route 53 latency and weighted records

If your web application is running on EC2 instances in more than one Region, and if you have more than one instance running in one or more of these Regions, you can use latency-based routing to route traffic to the correct region and then use weighted records to route traffic to instances within the region based on weights that you specify. To use latency and weighted records in Amazon Route 53 together:

- 1) Create a group of weighted records for your EC2 instances in each region.
 - a) Give each weighted record the same value for *Record Name* and *Record Type*.
 - b) For *Value/Route traffic to*, choose IP address or another value depending on the record type, and specify the value of one of the EC2 IP addresses.
 - c) If you want the EC2 instances to weigh equally, specify the same value for *Weight*.
 - d) Specify a unique value for *Set ID* for each record.
 - e) Associate a Route 53 health check for your instances under this weighted record.
- 2) If you have multiple EC2 instances in other regions, repeat Step 1 for the other regions, but specify a different value for *Name* in each region.
- 3) For each region in which you have multiple EC2 instances, create a latency alias record. For *Value/Route traffic to*, choose *Alias to another record in this hosted zone*, and specify the value of the *Record Name* of your weighted records in that region. Set the value of *Evaluate Target Health* to Yes.
- 4) For each region in which you have a single EC2 instance, create a latency record. For *Record Name*, specify the same value that you specified for the latency alias records created in Step 3. For *Value/Route traffic to*, choose *IP address or another value depending on the record type*, and specify the IP address of your EC2 instance in the Region.



Latency alias records allow you to utilize different regions that are close to your users for low latency performance. Together with weighted records, you ensure that your applications are protected from the failure of a single endpoint or an Availability Zone. Lastly, enabling *Evaluate Target Health* in your latency alias records lets Route 53 determine whether there are any healthy resources in a region before trying to route traffic there. If there are none, Route 53 chooses a healthy resource in the other region where you also have a latency alias record set up.

Route 53 weighted multi-record answers

A Route 53 weighted record can only be associated with one record, meaning a combination of one name and one record type. But it is often desirable to define weights for DNS responses that contain multiple records. In the event of an endpoint failure, providing multiple IP addresses in DNS responses provides users with alternative endpoints. You can even protect against the failure of an availability zone if you configure responses to contain a mix of IPs hosted in two or more availability zones.

These types of weighted multi-record answers can be achieved by using a combination of records and weighted alias records. You can group multiple endpoints into distinct record sets with each containing a subset of the IP addresses. You can then create a weighted alias record that points to each group and assign a corresponding weight. Weighted multi-record answer routing is different from Route 53 multi-value answer routing.

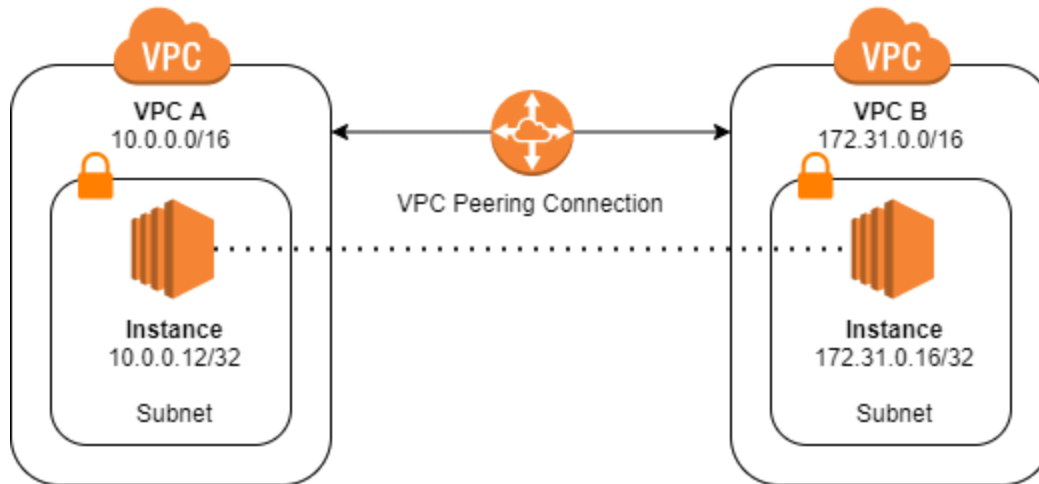
References:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/TutorialLBRMultipleEC2InRegion.html>
<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/TutorialWeightedFTMR.html>
<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover-complex-configs.html>

Longest Prefix Match: Understanding Advanced Concepts in VPC Peering

VPC Peering Basics

In AWS, a Virtual Private Cloud (VPC) peering connection is a networking connection between two VPCs which allows you to route specific traffic between them using either private IPv4 addresses or IPv6 addresses.



A VPC peering connection can be created between your own VPCs, or alternatively, a VPC in another AWS account. You can also create an inter-region VPC peering connection where the VPCs are located in different AWS Regions. Amazon EC2 Instances in either VPC can communicate with each other freely as if they are within the same network.

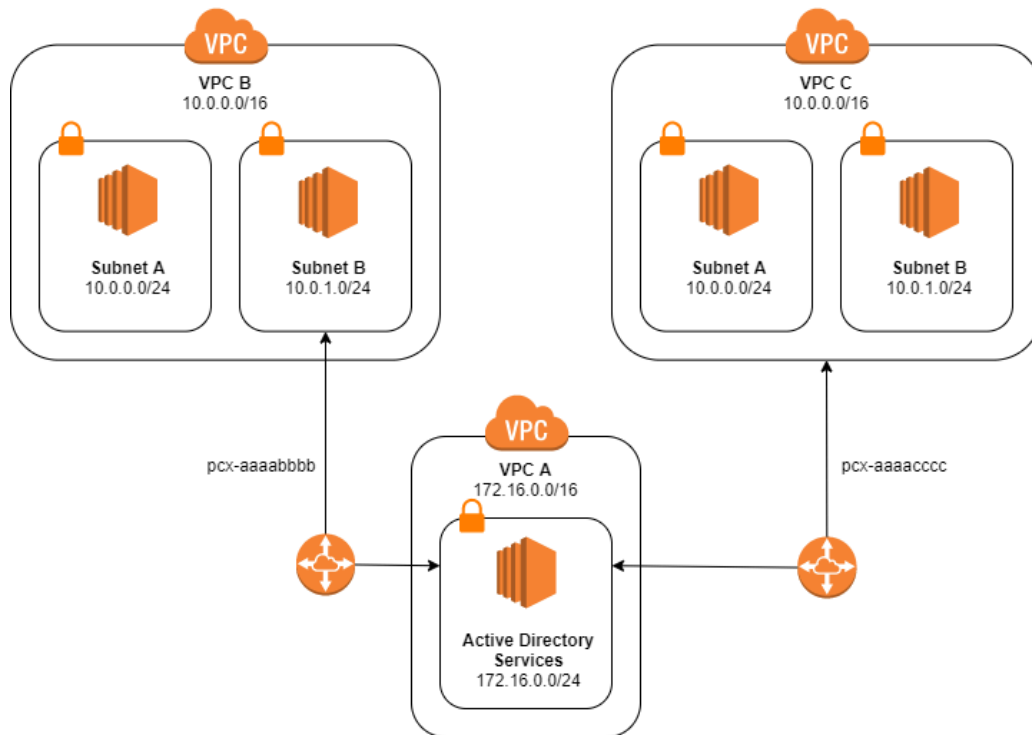
One example of VPC Peering is the integration of third-party services to your AWS account. Say you want to utilize a cloud database service from an external provider, like MongoDB Atlas which provides AWS, GCP, and Azure-backed clusters. In order for your EC2 instances to communicate with your external MongoDB cluster, you need to [establish a VPC Peering connection to the MongoDB Atlas VPC](#) first.

There are various VPC Peering setups that you can put up such as configurations with ClassicLink, configurations with routes to an entire CIDR block (VPCs don't have subnets) and lastly, configurations with **specific** routes (VPCs have two or more subnets). This article will focus on the latter type in which your VPC has peered with 2 VPCs and configured with specific routes on their route table, leveraging on the *longest prefix match* algorithm.

Longest Prefix Match – what is it?

As an overview, let say that you have a central VPC (labeled as VPC A below) with one subnet. This has a VPC peering connection between VPC A and VPC B (pcx-aaaabbbb) and also between VPC A and VPC C (pcx-aaaacccc).

The key point of this scenario is: Both VPC B and VPC C have **matching** CIDR blocks of 10.0.0.0/16 as shown in the diagram below:



At this point, if there is incoming traffic from VPC A which is intended to 10.0.0.66, which VPC will it go to? Will it be VPC B or VPC C, considering that these two VPCs have exactly the same prefix (CIDR block)? What's your guess?

This scenario is an advanced topic regarding VPC Peering where you have one VPC Peered with 2 VPCs in which it uses *Longest Prefix Match* for directing traffic based on your route table configuration. Let's go over the basics first in order for us to better understand this scenario:

The term "longest prefix match" is basically an algorithm used by routers in Internet Protocol (IP) networking used for choosing an entry from a forwarding route table. It is possible that each entry in a forwarding table may specify a **sub-network** in which one destination address may match more than one forwarding table entry. In this case, 10.0.0.0/24 is a sub-network of 10.0.0.0/16 CIDR block. At this point, the "**longest** prefix" actually refers to the one with the **longest** subnet mask which is the most specific of the matching table entries where the traffic will be forwarded.



Route Table	Destination	Target
VPC A	172.16.0.0/16	Local
	10.0.0.0/24	pcx-aaaabbbb
	10.0.1.0/24	pcx-aaaacccc
VPC B	10.0.0.0/16	Local
	172.16.0.0/16	pcx-aaaabbbb
VPC C	10.0.0.0/16	Local
	172.16.0.0/16	pcx-aaaacccc

Now that we get the context of “Longest Prefix Match”, we can now better understand how this works. Each VPC has a CIDR Block of 10.0.0.0/16 with two subnets: 10.0.0.0/24 (Subnet A) and 10.0.1.0/24 (Subnet C).

Adding an entry of **10.0.0.0/24** to pcx-aaaabbbb on your route table is the actual implementation of the prefix match we discussed earlier. Since 10.0.0.0/24 is a sub-network (Subnet A) of 10.0.0.0/16, we can better control the flow of traffic. The CIDR block of 10.0.0.0/24 has a total of 256 IP addresses with a range starting from 10.0.0.0 to 10.0.0.255.

The same goes for Subnet B which has a CIDR block of 10.0.1.0/24. Since its prefix is the same with Subnet A, it also has a total of 256 IP addresses with a range starting from 10.0.1.0 to 10.0.1.255 address.

Remember that the IP address in the question is 10.0.0.66 which is within the range of the 10.0.0.0/24 sub-network (Subnet A). Since we have a specific entry to pcx-aaaabbbb for this, the router’s behavior will forward the traffic to VPC B.

Reference:

<https://docs.aws.amazon.com/vpc/latest/peering/peering-configurations-partial-access.html#one-to-two-vpcs-lpm>

Automate your EBS Snapshots using Amazon Data Lifecycle Manager (Amazon DLM)

Amazon Data Lifecycle Manager brings a ton of convenience to customers who take regular EBS snapshots. You won't need to script your own Lambda function/Cloudwatch Event anymore just to backup your important files. Amazon DLM features a scheduler which you can configure to create a regular schedule for taking EBS snapshots. You can also define a retention period for EBS snapshots by creating lifecycle policies based on tags. Amazon DLM can back up select EBS volumes or all EBS volumes attached to an EC2 instance that has your specific tags. Any future EBS volume that needs to be included in your snapshot policy can be easily included by adding the identifying tag. What's more, this service is currently available at no additional cost.

To get started with Amazon DLM,

- 1) Navigate to the EC2 console, and click on **Lifecycle Manager** under **Elastic Block Store** on the left pane.
- 2) Select **Create Snapshot Lifecycle Policy**.
- 3) Enter a **description**, select if the resource type should be **EBS volumes** or **EC2 instances**, then enumerate the **tags** that would identify which resources will be included in your lifecycle policy.

[Policies](#) > Create Snapshot Lifecycle Policy

Create Snapshot Lifecycle Policy

Data Lifecycle Manager for EBS Snapshots will help you automate the creation and deletion of EBS snapshots based on a schedule. Volumes are targeted by tags

Description*
i

Select resource type
☒ Volume
☐ Instance

Target with these tags
This policy will be applied to EBS volumes with **any** of the following tags.

*
C

Lifecycle Policy Tags	
Key	Value
(128 characters maximum)	(256 characters maximum)

This resource currently has no tags

Add Tag
50 remaining (Up to 50 tags maximum)

- 4) You can also add your own tags to the lifecycle policy for easier identification.
- 5) Next, specify the IAM role that would allow the service to manage your volumes. AWS provides a default role, **AWSDataLifecycleManagerDefaultRole**, or you can create a custom IAM role.
- 6) You can define up to four (4) policy schedules. Each policy schedule describes how often snapshots are to be created by the policy, as well as the configuration for those snapshots.



▼ Policy Schedule 1

Schedules define how often snapshots are to be created by the policy, as well as the configuration for those snapshots. You must configure the default schedule for this policy. You can optionally configure up to three additional schedules for the policy.

Schedule name*	Schedule 1	
Frequency	Daily	
Every	12 Hours	
Starting at	09 : 00 UTC	
Retention type*	Count	
Retain*		

Tagging information (optional) This applies only to snapshots created by DLM in the same region. Tagging options for snapshots copied by DLM are available in the cross region copy section.

Tag created EBS snapshots Any snapshot created with this policy will automatically be tagged with the policy ID and schedule name.

Copy Tags from volume ☐

Additional tags	
Key	(128 characters maximum)
Value	(256 characters maximum)

This resource currently has no tags

- Add your **Schedule name**
 - Define the **frequency**. Valid values are daily, weekly, monthly, yearly, or a custom cron expression.
 - The **Retention Type** specifies how often snapshots are cleaned. Values are count and age. Use count if there is a max number of snapshots that you would like to keep. Use age if there is a lifespan for your snapshots. You can copy each snapshot to up at three additional Regions.
- You also have the option to enable **Fast Snapshot Restore** and **Cross Region Copy**. If you enable fast snapshot restore, you must choose the Availability Zones in which to enable it. You are billed for each minute that fast snapshot restore is enabled for a snapshot in a particular Availability Zone.
 - You may review your configuration in the **Policy Summary** section. Once everything is good to go, click **Create Policy**.

References:

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/snapshot-lifecycle.html>



Real-time Log Processing using CloudWatch Logs Subscription Filters

You use CloudWatch Logs subscriptions to gain access to a real-time feed of log events from CloudWatch Logs and have it delivered to other services such as an Amazon Kinesis stream, an Amazon Data Firehose, or AWS Lambda. A **subscription filter** defines the filter pattern to use for filtering which log events get delivered to your AWS resource, as well as information about where to send matching log events to. The logs sent to the destination resource are Base64 encoded and compressed with the gzip format. A log group can have two subscription filters at most.

Key elements of a subscription filter include:

- **log group name** - indicates which log group is associated with the subscription filter.
- **filter pattern** - defines how CloudWatch Logs interpret the data for each log event. The filtering expression will control what is delivered to the destination service.
- **destination arn** - could be the ARN of a Kinesis stream, a Data Firehose stream, and/or a Lambda function.
- **role arn** - to put the data in the chosen destination, you must create an IAM Role and grant CloudWatch Logs the necessary permissions.
- **distribution** - this element is only applicable for Amazon Kinesis Stream. By default, log data is grouped by a log stream. This log data can be distributed more evenly to your Kinesis stream by grouping them at random.

Cross-Account Log Data Sharing with Subscriptions

To collaborate with a different AWS account, you can use *cross-account data sharing*. It will allow you to receive the log events from their accounts to your AWS resources. To start receiving log events from cross-account users, the log data recipient must first create a CloudWatch Logs destination. The log group and destination must be in the same Region, but the AWS resource that the destination points to can be in a different region. Lastly, the only supported destination resource for cross-account subscriptions is Kinesis Streams.

There are two parties involved in cross-account data sharing:

- **Log data sender** - retrieves the destination information from the recipient and prepares CloudWatch Logs to send its log events to the specified destination.
- **Log data receiver** - sets up a CloudWatch Logs destination that encapsulates a Kinesis stream and prepares CloudWatch Logs to receive log data. The recipient then shares the information about his destination with the sender.



Key elements of the destination:

- **Destination name** - a user friendly identifier of the destination.
- **Target ARN** - the ARN of the destination resource for the subscription feed.
- **Role ARN** - grants CloudWatch Logs the necessary permissions to put data into the chosen Kinesis stream.
- **Access policy** - an IAM policy that defines who is allowed to send log data to the recipient's destination.

References:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/Subscriptions.html>

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/SubscriptionFilters.html>

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/CrossAccountSubscriptions.html>

Scaling Memory-Intensive Applications in AWS

Scaling is an important practice in AWS to make sure your applications are always available to meet demand. To scale an EC2 instance based on the CPUUtilization metric, we can rely on Amazon CloudWatch for metric monitoring since the metric is available by default for EC2 instances. However, if the EC2 instance is memory-intensive and needs to scale based on MemoryUtilization, we will need to install a CloudWatch agent on the instance first. The CloudWatch agent will collect system-level metrics such as memory and store them in Amazon CloudWatch Logs.

A CloudWatch agent is different from the SSM agent. The CloudWatch agent allows you to collect more system-level metrics from your EC2 and on-premises servers than just the standard CloudWatch metrics; while SSM Agent processes requests from the AWS Systems Manager and configures your machine as specified in the request. Although you can configure a document that runs a script to log system-level metrics and have the SSM agent perform this task, it is not a very efficient solution. Instead of using SSM Agent to gather log files on each instance, you can use the Amazon CloudWatch agent to collect additional metrics and logs for you at a more convenient process.

An Auto Scaling Group uses a launch template to determine how it will provision additional EC2 instances for you. Remember that you can only associate one launch template at a time and you are also not allowed to modify an existing launch template settings. If you wish to update your autoscaling group's launch template settings, you must create a new launch template from scratch or copy and modify the existing launch template, then you modify the ASG to use the newly created launch template.

An example scenario:

An organization is running a memory-intensive application on compute-optimized instances. The instances are launched through an autoscaling group and have Cloudwatch Agent installed already. When the workload increases, the autoscaling group is configured to scale based on CPU usage. They noticed that the performance



of the application was still slow so they decided to increase the number of instances further. From a Solutions Architect's perspective, this is not the best approach since they are scaling on the wrong metric. To improve the application's performance while saving costs, they should instead create a new launch template and use a memory-optimized instance as the instance type. Afterwards, they should modify the scaling policy to scale based on memory usage.

References:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/metrics-collected-by-CloudWatch-agent.html>

https://docs.amazonaws.cn/en_us/systems-manager/latest/userguide/monitoring-cloudwatch-agent.html

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/change-launch-config.html>

Using AWS Step Functions For Orchestrating Serverless Workflows

Step Functions lets you orchestrate the components of an application into a serverless workflow. A **workflow** is a series of steps, with the output of one step acting as input into the next step. A workflow defines the process of how your application accomplishes its goal from start to end.

In Step Functions, we refer to the workflow as **State machines**, and each step in a State machine is called a **state**.

A State Machine is written using the Amazon States Language or ASL, which has a similar syntax to a JSON document. If you don't want to write in ASL, Step Functions also provides a drag-and-drop interface that further simplifies how you build your state machine. Step Functions will write the state machine definition for you.

States

There are eight state types that you can use to build a State machine:

- **Task state** - represents a single unit of work in your State machine. Application code that needs to be executed by your Lambda function is done by the Task state. A Task state can only include a single Lambda function, an ECS task, an SNS notification, a DynamoDB action, etc.
- **Choice state** - adds a branching logic to your workflow. You can think of it as an if-then-else statement where there is more than one possible outcome that can occur when evaluating a parameter against a condition that you set.
- **Parallel state** - performs two branches of execution at the same time. A use case for this is when you want to run two independent activities that perform different processes to an input from the same node.
- **Wait state** - adds a delay to your State machine before it continues executing states. This is similar to how a Sleep function in Python works.



- **Fail State** - stops the execution of the state machine and marks it as a failure. One example where you could use this is when you have a registration process, and you want to deny any invalid form that a user submitted. You can mark the registration as a failure and have a custom error message like 'InvalidUsername'.
- **Succeed State** - stops the execution successfully.
- **Pass state** - can use the Pass state to pass the input from the previous state to its output without performing work. This is useful when you're debugging a portion of your state machine and you want to have visibility of the output of a particular state.

Rearchitecting Serverless Applications

When it comes to serverless applications, AWS Lambda often stands out as the first choice due to its flexibility and ease of use. It's the go-to solution for executing code in response to events without the need to manage servers. However, Lambda's 15-minute execution limit can be a bottleneck for more extended operations. This limitation becomes apparent in scenarios like processing application forms, where sequential steps such as parsing data, validating information, and performing background checks are required. These operations demand a more flexible approach to handle the complexity and duration of the tasks involved.

AWS Step Functions offers a robust solution for orchestrating workflows that exceed Lambda's time constraints. By leveraging Step Functions, you can design workflows that split the logic into multiple Lambda functions, each handling a part of the process. This approach not only circumvents Lambda's execution limit but also organizes the workflow into a series of discrete, manageable tasks. Optionally, you can use AWS Step Functions' integration with AWS SDK to call API calls directly without the need for additional Lambda functions as intermediaries for API calls.

Built in retry and errors

Furthermore, Step Functions offers built-in error handling and retry mechanisms. If the background check API fails or times out, Step Functions can automatically retry the task according to predefined policies. Once all checks are completed, Step Functions can update the application status in the database, either directly or through another Lambda function, depending on the task's complexity and requirements.

References:

<https://docs.aws.amazon.com/step-functions/latest/dg/concepts-states.html>

<https://aws.amazon.com/blogs/aws/new-aws-step-functions-workflow-studio-a-low-code-visual-tool-for-building-state-machines/>



AWS Pricing Models

Lowering the customers' overall infrastructure and management costs is always the objective of an AWS Solutions Architect. When we talk about reducing costs, we do not mean that we will compensate performance, security, and availability just to reach that lower level of spending. The process is much more complex than that. When we want to lower costs, we as Solutions Architects will find ways to optimize and refine the existing infrastructure. We weigh all of our available options and design new solutions that are centered around those options. We make sure that these designs still achieve the business objectives, performance baselines, security requirements, and all of the important bits while bringing in higher savings for our customers *in the long term*. That is why, to become an AWS Solutions Architect Professional, you must be aware of all the pricing models in AWS.

In AWS, there are many pricing models to choose from. You have:

- 1) Pay-as-you-go or on-demand
- 2) Long term capacity reservations
- 3) Purchase based on available capacity or Spot
- 4) Volume discounts

There are three fundamental drivers of cost with AWS: compute, storage, and outbound data transfer. For compute capacity, you often associate it with the first three models. For storage and data transfers, you associate them with the last model.

Amazon EC2 and services that use EC2 as the backend, such as Amazon ElastiCache, Amazon RDS, and Amazon Redshift, offer greater savings if you purchase Reserved Instances rather than On-Demand Instances. A reservation allows you to commit to one full year or three full years of continuous usage. You may also choose to pay a portion of the total bill or the full bill upfront, which will give you even higher discount rates. There are many ways to reserve compute capacity in AWS, each with their own advantages and disadvantages. Though one thing is for certain, if you expect workloads to run continuously for long periods of time then you should always consider using Reserved Instances.

Conversely, if you have workloads running only for short bursts or tasks that are only handled as they arrive in a queue, then consider using Spot Instances. Spot Instances let you take advantage of unused EC2 capacity in the AWS cloud, and you just pay the Spot price that's in effect for the current hour for the instances that you launch. Since you can never expect when your Spot instances will be terminated, only use them if your workload can handle interruptions. Another way to use Spot instances is when you just need "extra capacity", such as stateless servers moving data from one place to another. You can maintain a fixed amount of compute capacity that's always running and have spot reduce the burden when there is a heavier workload.

i Solutions Architect Professional Exam Notes:

What about using mixed pricing models?

In the exam, you will encounter questions that ask you how to provision your compute resources in the most cost-effective way. A common service to use as an example is Amazon Redshift. You can combine Reserved, On-Demand, and Spot instances in one cluster, and delegate each model according to its own area of strength. For example, you can have one reserved leader node and a mix of on-demand and spot compute nodes.

Lastly, when you are managing multiple accounts through AWS Organizations, you can benefit from lower total costs through volume discounts. Some services, such as AWS Data Transfer Out and Amazon S3, have volume pricing tiers across certain usage dimensions that give you lower prices the more you use the service. So with consolidated billing, AWS considers the usage across all accounts for the pricing tier eligibility.

References:

https://d1.awsstatic.com/whitepapers/aws_pricing_overview.pdf

<https://d1.awsstatic.com/whitepapers/cost-optimization-reservation-models.pdf>

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-purchasing-options.html>

<https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/useconsolidatedbilling-discounts.html>

Reserved Instances and Savings Plan

As mentioned in the previous section, there are many options to save costs with Amazon EC2 instances. You choose the arrangement that works best for your needs.

Purchasing Reserved Instances is the commonly known method of receiving discounts for EC2 pricing. You select how many years you would like to use the reserved instance and specify if you will be paying an upfront cost for greater discount benefits. Services that support RI include Amazon EC2, Amazon RDS, Amazon ElastiCache, and Amazon Redshift. There are two types of reserved instances to choose from:

1) Standard RI

- Can be purchased to apply to instances in a specific Availability Zone (zonal Reserved Instances), or to instances in a Region (regional Reserved Instances).
- Enables you to modify instance attributes such as Availability Zone, scope (from zonal to regional and vice versa), network platform (EC2 Classic to VPC and vice versa), and instance size (**within the same instance type e.g. C-type**) of your Reserved Instance.
- Standard Reserved Instances typically provide the highest discount levels.

2) Convertible RI

- Can be purchased to apply to instances in a specific Availability Zone (zonal Reserved Instances), or to instances in a Region (regional Reserved Instances).



- Enables you to exchange one or more Convertible Reserved Instances for another Convertible Reserved Instance **with new attributes**. These attributes include instance family, instance type, platform, scope, and tenancy, if the exchange results in the creation of a Reserved Instance of equal or greater value.
- Useful when workloads are likely to change or you do not have forecasted data on your usage patterns.

Purchase Reserved Instances

☐ Only show offerings that reserve capacity

Platform Linux/UNIX

Tenancy Default

Offering Class Any

Instance Type t2.micro

Term Any

Payment Option Any

Search

Seller	Term	Effective Rate	Upfront Price	Hourly Rate	Payment Option	Offering Class	Quantity Available	Desired Quantity	Normalized units per hour	
AWS	12 months	\$0.007	\$0.00	\$0.007	No Upfront	standard	Unlimited	1	0.5	Add to Cart
AWS	12 months	\$0.008	\$0.00	\$0.008	No Upfront	convertible	Unlimited	1	0.5	Add to Cart
AWS	36 months	\$0.005	\$0.00	\$0.005	No Upfront	standard	Unlimited	1	0.5	Add to Cart
AWS	36 months	\$0.006	\$0.00	\$0.006	No Upfront	convertible	Unlimited	1	0.5	Add to Cart
3rd Party	6 months	\$0.007	\$15.00	\$0.003	Partial Upfront	standard	1	1	0.5	Add to Cart
AWS	12 months	\$0.007	\$30.00	\$0.003	Partial Upfront	standard	Unlimited	1	0.5	Add to Cart
AWS	12 months	\$0.008	\$34.00	\$0.004	Partial Upfront	convertible	Unlimited	1	0.5	Add to Cart
AWS	12 months	\$0.007	\$59.00	\$0.000	All Upfront	standard	Unlimited	1	0.5	Add to Cart

You currently have no items in your cart.

Cancel

View Cart

Savings Plans is a fairly new offering from AWS that works similar to RIs. They give you pricing discounts in exchange for a long-term usage commitment. Savings Plans is not only limited to EC2 instances, but can also be applied to Lambda and Fargate usage. AWS offers two types of Savings Plans:

1) Compute Savings Plans

- Provide the most flexibility and help to reduce your costs by up to 66%.
- These plans automatically apply to EC2 instance usage **regardless** of instance family, size, AZ, region, OS or tenancy, and also apply to Fargate and Lambda usage.
- For example, you can change from C4 to M5 instances, shift a workload from EU (Ireland) to EU (London), or move a workload from EC2 to Fargate or Lambda at any time and automatically continue to pay the Savings Plans price.

2) EC2 Instance Savings Plans

- Provide the lowest prices, offering savings up to 72% in exchange for commitment to usage of **individual instance families** in a region (e.g. M5 usage in N. Virginia).

- This automatically reduces your cost on the selected instance family in that region regardless of AZ, size, OS, or tenancy.
- EC2 Instance Savings Plans give you the flexibility to change your usage between instances within a family in that region. For example, you can move from c5.xlarge running Windows to c5.2xlarge running Linux and automatically benefit from the Savings Plans prices.

Purchase Savings Plans [Info](#)

Savings Plans are a flexible pricing model that offer low prices on AWS usage, in exchange for a commitment to a consistent amount of usage (measured in \$/hour) for a 1- or 3-year term.

Purchase details [Info](#)

Savings Plan type

- ☒ **Compute Savings Plans**
Applies to EC2 instance usage, AWS Fargate, and AWS Lambda service usage, regardless of region, instance family, size, tenancy, and operating system.
[Learn more](#) 

- ☐ **EC2 Instance Savings Plans**
Applies to instance usage within the committed EC2 family and region, regardless of size, tenancy, and operating system.
[Learn more](#) 

Term

- ☒ 1-year
☐ 3-year

Purchase commitment [Info](#)

Hourly commitment

Your hourly commitment at Savings Plan rates. To maximize your savings, see our [recommendations](#).

Enter hourly commitment amount (USD)

Payment option

- ☒ All Upfront
☐ Partial Upfront
☐ No Upfront

Figure: AWS Compute Savings Plans

Savings Plan type

☐ Compute Savings Plans

Applies to EC2 instance usage, AWS Fargate, and AWS Lambda service usage, regardless of region, instance family, size, tenancy, and operating system.

[Learn more](#)

☒ EC2 Instance Savings Plans

Applies to instance usage within the committed EC2 family and region, regardless of size, tenancy, and operating system.

[Learn more](#)

Term

☒ 1-year

☐ 3-year

Region

US East (N. Virginia)

Instance Family

c5

Purchase commitment [Info](#)

Hourly commitment

Your hourly commitment at Savings Plan rates. To maximize your savings, see our [recommendations](#).

Enter hourly commitment amount (USD)

Payment option

☒ All Upfront

☐ Partial Upfront

☐ No Upfront

Figure: AWS EC2 Instance Family Savings Plans



Savings Plans also works with AWS Organizations, similar to RIs. By default, the benefit provided by Savings Plans is applicable to usage across all accounts within an AWS Organization/Consolidated billing family. However, you can also choose to restrict this benefit to only the account that purchased them. Your RIs will continue to work alongside Savings Plans if you wish to use both.

A few notable differences between Savings Plans and Reserved Instances are:

- You cannot directly reserve compute capacity with Savings Plans. You can, however, reserve capacity with On Demand Capacity Reservations and pay lower prices on them with Savings Plans.
- You also cannot sell unused Savings Plans capacity in the AWS Marketplace, unlike RIs.

References:

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/reserved-instances-types.html>

<https://docs.aws.amazon.com/whitepapers/latest/cost-optimization-reservation-models/standard-vs.-convertible-offering-classes.html>

<https://aws.amazon.com/savingsplans/>

Amazon EC2 Auto Scaling

Amazon EC2 Auto Scaling helps maintain application availability by automatically adding or removing EC2 instances based on workload demand. It improves fault tolerance, performance, and cost optimization by ensuring the correct number of instances are running at all times.

With Amazon EC2 Auto Scaling, you can configure scaling policies that automatically adjust capacity according to metrics such as CPU utilization, network traffic, or custom CloudWatch metrics.

Common benefits include:

- Improved application availability
- Automatic scaling during traffic spikes
- Reduced operational overhead
- Cost savings by removing unused capacity
- Better fault tolerance across Availability Zones

Dynamic Scaling

Dynamic scaling automatically adjusts the number of EC2 instances in response to real-time demand.

Amazon EC2 Auto Scaling continuously monitors Amazon CloudWatch metrics and scales resources in or out whenever thresholds are breached.

There are several dynamic scaling policy types:



Scaling Type	Description
Target Tracking Scaling	Maintains a specific metric target such as 50% CPU utilization
Step Scaling	Adds or removes instances based on alarm thresholds
Simple Scaling	Performs a scaling action after a CloudWatch alarm triggers

Example:

- Add 2 EC2 instances when CPU utilization exceeds 70%
- Remove 1 EC2 instance when CPU utilization drops below 30%

Dynamic scaling is ideal for workloads with unpredictable or fluctuating traffic patterns.

Scheduled Scaling

Scheduled scaling allows scaling actions to occur at predefined times.

This is useful when traffic patterns are predictable, such as:

- Business hours
- Weekly events
- Payroll processing
- Marketing campaigns

Administrators define schedules that increase or decrease capacity automatically.

Example:

- Increase capacity to 10 instances every weekday at 8:00 AM
- Reduce capacity to 2 instances every weekday at 8:00 PM

Scheduled scaling helps ensure sufficient resources are available before expected traffic increases occur.

Predictive Scaling

Predictive scaling uses machine learning to forecast future traffic patterns and proactively launch EC2 instances before demand increases.

Amazon EC2 Auto Scaling analyzes historical utilization data and predicts future capacity requirements. This helps applications maintain performance during predictable traffic spikes.



Predictive scaling is particularly useful for applications with recurring usage patterns such as:

- Daily traffic peaks
- Weekly workloads
- Seasonal demand
- E-commerce events

Instead of reacting to traffic increases after they occur, predictive scaling launches instances in advance to reduce latency and improve user experience.

Example:

An e-commerce platform experiences increased traffic every evening at 7:00 PM. Predictive scaling learns this pattern and automatically launches additional EC2 instances before the traffic spike begins.



Using Different AWS Cost Management Services

Your AWS Billing is one thing that you should always keep an eye out for. One of the best benefits the cloud can give you is lowering your CapEx so you can direct it to other more valuable targets. Therefore, you should also be knowledgeable in the different cost monitoring and cost control services in AWS. These services will be your main go-to tools to monitor your spending, as well as to discover opportunities to further reduce costs.

1) AWS Cost Allocation Tags

AWS Cost Allocation Tags allow you to generate cost allocation reports as CSV files with your usage and costs grouped by your active tags. Cost tagging is a very useful strategy if you need to group different AWS resources according to defined categories, such as business cases for example. It also allows you to check if the performance gains from your usage are proportional to the amount you are paying for. This allows you to make informed decisions on how to manage your own AWS resources cost-effectively.

2) AWS Cost Explorer + AWS Cost and Usage Report

AWS Cost Explorer and AWS Cost and Usage Report are two very commonly used services for understanding your spending in AWS. Visualization is the key aspect of these two services. They allow you to construct meaningful diagrams that easily show you the trends in your spending. You can filter through the data by specifying your parameters (such as which services to include, which regions, which time period, etc), and this can provide you either a high level overview or a more granular view of your spending patterns. AWS Cost Explorer also analyzes your billing history and provides you a forecast of your future costs and usages. Lastly, once you have configured a cost and usage dashboard that you would like to review continuously, you can save it as a report and return to it anytime you wish. This is very useful if you like to track your EC2 Reserved Instance usage for example.

3) AWS Budgets + Amazon SNS

Once you have a pretty good idea of your cost and usage patterns in AWS, you can start configuring your own AWS Budgets alerts to make sure you don't overspend. Having an alert watching over your spending, especially when you manage multiple accounts, will give you the peace of mind that you need. The alerts also work with Amazon SNS to notify relevant personnel when your budget is about to be exceeded. This gives you and your business the opportunities to refine your operations and remove any unnecessary resource consumption. Budget tracking may also reflect business growth, since the higher your budget is, the more you can innovate and expand your operations.

4) AWS Trusted Advisor



AWS Trusted Advisor is an indispensable tool for ensuring your account is as cost-effective as possible. The Cost Optimization feature under AWS Trusted Advisor makes use of the well-architected best practices for cost-efficiency, so you have a centralized monitoring solution that continuously reviews your account for any items that can incur you unnecessary expenses. How AWS Trusted Advisor does this is by having multiple checks that scan for underutilized (e.g. idle instances) and unoptimized (e.g. oversized instances) resources that are running in your account. The number of Trusted Advisor checks that will be available to you will depend on your support plan. Nevertheless, you should often review your AWS Trusted Advisor to ensure all your resources are well-utilized and right-sized.

5) Consolidated Billing for AWS Organizations

If you are running AWS Organizations, you should enable consolidated billing to enjoy some of the benefits of volume discounts. Services such as S3 and Data Transfer Out offer pricing tiers that lower the cost the more you use the service. Aside from volume discounts, if you are the master account (payer account), you should also centrally track and monitor spending of each of your payee accounts. Do note, however, that if you do not have all AWS Organizations features enabled, you cannot control how each account handles their spending. The consolidated billing feature treats all the accounts in the organization as one account. Therefore, all accounts in the organization can receive the hourly cost benefit of Reserved Instances that are purchased by any other account. Reserved Instance discount sharing can be disabled if you do not wish to share RIs.

References:

<https://aws.amazon.com/aws-cost-management/>

<https://docs.aws.amazon.com/pdfs/whitepapers/latest/cost-management/cost-management.pdf>